

---

# LSTM Performance Limits in Neural Decoding

---

Daniel Kofman

## Abstract

This project builds upon the results of prior work: “Benchmarking Deep Learning Architectures for Predicting Visual Stimuli Given Single Neuron Spike Patterns”, which found that among the 10 different machine learning methods explored, a simple LSTM performed the best on visual stimuli classification using neural firing rate data from mice [Carpenter et al., 2024]. The first goal of this project was to streamline, debug, and validate the results of this previous work, focusing on data processing, data exploration, and the LSTM model. This refactored pipeline would increase transparency, making verification of results by third parties more feasible and computationally efficient. During this process average LSTM test accuracy across all 32 mice was increased by 6.43%. A natural second goal was to investigate why LSTM classification performance varied so widely across mice, and if low accuracies could be boosted to match higher accuracy mice. What I found was evidence of an inherent true signal capacity limit in the data itself that made significant accuracy improvements difficult, even for low accuracy mice. This was supported by the finding that roughly 78% of the variation in performance accuracy was explained by the average neural firing rate of each mouse (calculated across all its neurons and across the entire time period of its recording session).

## 1 Introduction

The ability to decode external visual stimuli from internal neural activity represents a significant frontier in both neuroscience and artificial intelligence. As recording technologies like Neuropixels probes evolve to capture large-scale spiking activity with high spatial and temporal resolution, the challenge shifts to developing computational models capable of interpreting this rich data. From biological understanding of information processing in animal brains, we know that a lot of information processing/encoding has already taken place, represented by the firing patterns of the measured neurons. Given this pre-processed information rich data, the challenge is to model how this information interacts and evolves over time.

The general problem of this project and Carpenter’s is classification of visual stimuli presented to mice based on their neural activity while observing that stimuli, recorded by surgically implanted probes. This project extends the work of Carpenter’s, the primary results of which were that when evaluating the performance of 10 different machine learning architectures: Long Term Short Memory, Principal Component to MVLR, Transformer, Single Layer NN, Spatial Temporal GAT, Support Vector Classifier, Graph Attention Network, Multivariate Logistic Regression, Multi-layer Perceptrons, and Graph Convolutional Network (ordered from best performance to worst), a standard 1-layer LSTM performed the best, achieving the highest classification accuracy score of 97.73% (among the best performing subject mice of each model) [Carpenter et al., 2024]. Additionally, performance varied quite drastically across mice, with the lowest performing mouse at 0.48%. The goals of this project are to make the results of Carpenter et al. more easily reproducible, and investigate what is causing the LSTM to perform much better on some mice over others.

## 2 Methodology

### 2.1 Dataset

I utilize data from the Allen Brain Observatory – Neuropixels Visual Coding experiment. The dataset comprises recordings from 32 mice, each subjected to a session where 118 natural images were flashed for 250 milliseconds, repeated 50 times per image [AIBS, 2019]. The spike activity was processed into time-binned firing rate counts (5 bins per 250 millisecond training example), resulting in ordered a time-series where each row corresponds to one time bin, grouped in blocks of 5 corresponding to one training example. Each column except for the last represents a 'unit', and the last column corresponds to the associated frame (numerically encoded). The physical probes into the mice's brain are not precise enough to measure individual neurons, rather they collect data on brain regions from which individual 'unit' spike activity is mathematically derived based on unique waveform signatures [AIBS, 2019], which are meant to correspond to neurons. This two dimensional dataset is split into predictor data and labels ( $X$  and  $y$ ), and  $X$  is reshaped into a 3-dimensional tensor with dimension:

$$(\# \text{ image exposure (samples)} \times \# \text{ time bins} \times \# \text{ neurons (varies per mouse)}) \quad (1)$$

$y$  remains a one-dimensional label vector. The reshaped dataset is then split into 72% training, 20% test, and 8% validation. Crucially, the model has to be trained on each mouse separately to account for inter-subject variability in neural topology.

### 2.2 Analysis

All notable changes made during the refactoring process were documented in the changelog file of this repository: <https://github.com/dkofma60/Neuropixels-Pipeline-Refactor>. The first key change was increasing data processing speed by 2.38x. The second key change was fixing a data splitting bug that 1) caused time bins from different training examples and their labels to become scrambled in testing and validation sets for mice that did not have exactly 50 training examples for each of the 118 visual stimuli, 2) didn't shuffle training examples when splitting data into test and validation sets (the last 20% of the time series samples were used for test, and the last 10% of the remaining 80% for validation) which introduced unnecessary bias. This only ended up affecting one mouse but significantly affected its performance, as shown in the results section. Lastly, 3) didn't stratify the splits by visual stimuli label, another source of unnecessary bias.

All experiments were done using 5 time bins per visual stimuli exposure, which was identified by Carpenter et al. to result in the best LSTM performance. Four methods were explored to boost LSTM performance: 1) normalizing firing rates across neurons, 2) weight decay, 3) decreasing LSTM hidden dimension, and 4) dropout on input and output layers.

When searching for an explanation of the vast LSTM performance variability across mice, 6 variables were examined: mouse age, mouse neuron count, mouse genotype, mouse sex, average firing rate across all neurons and time bins, and neuron firing rate standard deviation averaged across neurons.

## 3 Results

### 3.1 Debugging

Section 3.9.2 of "src/model\_predictions" in the project repository walks through an example of the effects of the baseline data splitter validation/test set scrambling bug.

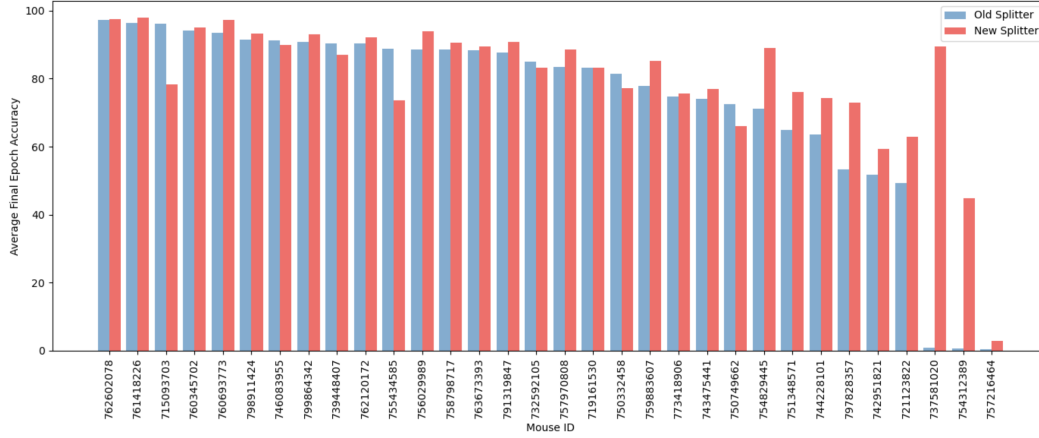


Figure 1: Old vs New Splitter Accuracy (Accuracies averaged over 3 runs per mouse).

Over 32 different mice, the new splitter performed on average 6.43% better. This improvement was primarily driven by mice 737581020 and 754312389, who saw accuracy improvements of 88.55% and 44.14% respectively. Interestingly, these were the only mice with substantial performance improvements, whereas all mice should be affected by the validation/test set scrambling bug. This motivated the next investigation to determine how many mice were actually affected by this specific issue.

```

762602078 rows mod (5*5): 0
761418226 rows mod (5*5): 0
715093703 rows mod (5*5): 0
760345702 rows mod (5*5): 0
760693773 rows mod (5*5): 0
798911424 rows mod (5*5): 0
746083955 rows mod (5*5): 0
799864342 rows mod (5*5): 0
739448407 rows mod (5*5): 0
762120172 rows mod (5*5): 0
755434585 rows mod (5*5): 0
756029989 rows mod (5*5): 0
758798717 rows mod (5*5): 0
763673393 rows mod (5*5): 0
791319847 rows mod (5*5): 0
732592105 rows mod (5*5): 0
757970808 rows mod (5*5): 0
719161530 rows mod (5*5): 0
750332458 rows mod (5*5): 0
759883607 rows mod (5*5): 0
773418906 rows mod (5*5): 0
743475441 rows mod (5*5): 0
750749662 rows mod (5*5): 0
754829445 rows mod (5*5): 0
751348571 rows mod (5*5): 0
744228101 rows mod (5*5): 0
797828357 rows mod (5*5): 0
742951821 rows mod (5*5): 0
721123822 rows mod (5*5): 0
737581020 rows mod (5*5): 10
754312389 rows mod (5*5): 0
757216464 rows mod (5*5): 0

```

Figure 2: Bins Mod 25 Across Mice.

I found the reason most mice did not have substantial improvements with the new splitter (which would indicate exposure to the validation/test set scrambling bug with the old splitter) was because the number of total bins (rows) were all divisible by 25, except for mouse 737581020. If the number of total bins is divisible by 5 (which is always the case because there are 5 bins for each frame exposure), then an 80/20 split will always cleanly split into 5 chunks of bins. Then if each of these individual chunks is also divisible by 5, that means the clean 80/20 split at the chunk level will also be a clean split at the frame exposure level, because each of these chunks will have an integer amount of groups of 5 bins.

Upon further inspection, it turns out that mouse 737581020 was seemingly the only mouse where the number of exposures per frame were stochastically generated, all other mice have exactly 50 exposures per frame.

The substantial improvement of mouse 754312389 with the new splitter is still unexplained by this analysis, but could be due to outsized impact of issues 2) and 3) of the data splitting bug as explained in the methodology section. In regards to 2), one could imagine mouse 754312389 being in a specific mental state during the first 80% of its session, causing a certain consistent neural activation pattern in response to the visual stimuli, and then shifting to a different mental state during the last 20% of its session, resulting in a different neural activation pattern. The model is only trained on the first 80% of the session with the baseline splitter, causing it to learn a pattern that does not repeat in the testing set. In regards to 3), since frame exposures are random throughout the session, it could be possible that for this mouse, for an unusual number of frames, their exposures tended to mostly occur in the last 20% of the session and not the first 80% the model was trained on, resulting in poor performance for those frames.

### 3.2 Accuracy Boosting

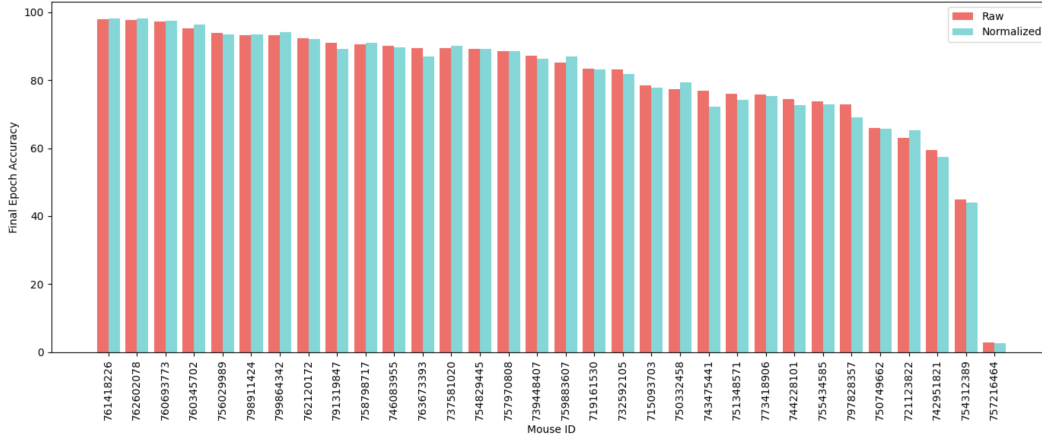


Figure 3: Raw vs Normalized Accuracy (averaged over 3 runs per mouse using new splitter).

For each of the 32 mice, new datasets were created by processing the existing 5 time-binned files into normalized (by neuron i.e. column) versions. The LSTM was then trained on these and compared to the raw-spike-trained LSTM. Differences are minimal, and if anything normalizing results in slightly worse performance.

A consistent pattern emerging across training runs was overfitting, as 100% training accuracy was reached in early epochs with test performance flattening out correspondingly quite quickly. To address this, a number of methods were attempted: weight decay, reduction in hidden dimension size, and dropout in the input and output layers. The goal for all 3 methods was to reduce the capacity of the model to memorize noise in the training data. None proved effective in any significant way, which specifically for hidden dimension size was unsurprising as this hyperparameter was already tuned in previous work [Carpenter et al., 2024].

### 3.3 Accuracy Gap Causation

The failure of regularization techniques to close the accuracy gap across mice (2.91% to 97.98%) leads to the suspicion that this gap may be explained by a feature/s inherent to the mice rather than a quirk in model design. Various regression and ANOVA models were built to determine if metadata could be used to predict LSTM accuracy across mice with statistical significance, and thus explaining the cause of the gap. Statistical significance will be defined as  $p < 0.05$ .

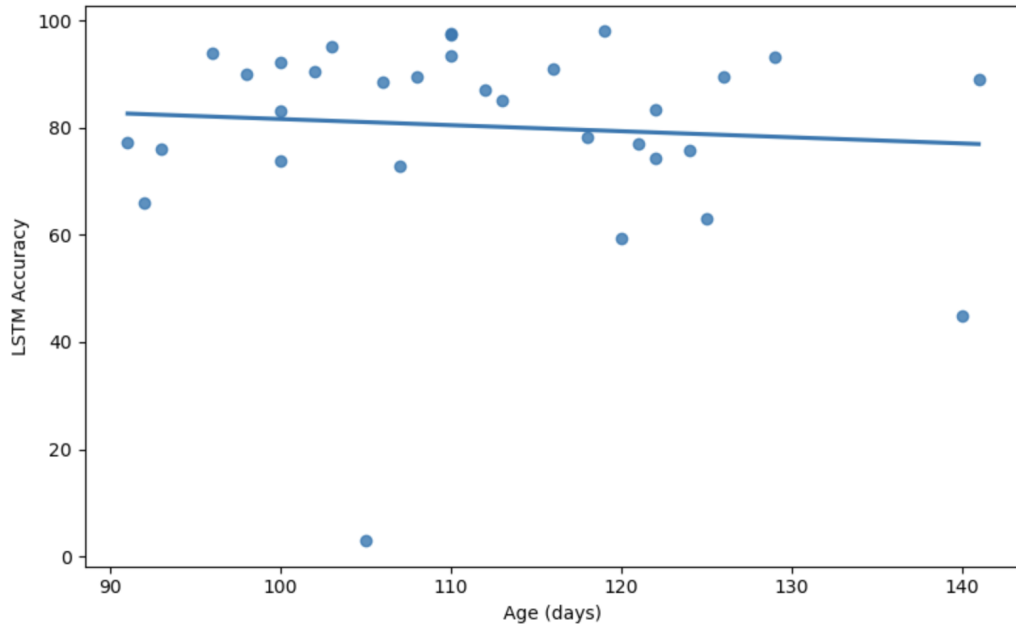


Figure 4: Accuracy vs Age.

The p-value of this simple linear regression model's overall significance F-test is 0.67, so mouse age likely doesn't explain the gap.

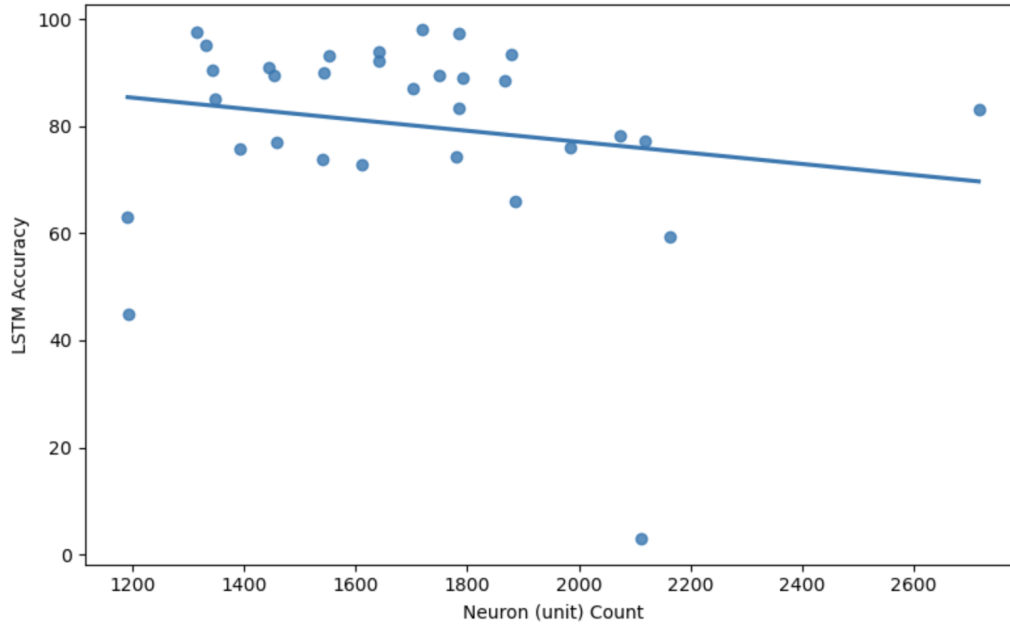


Figure 5: Accuracy vs Neuron Count

The p-value of this simple linear regression model's overall significance F-test is 0.32, so mouse neuron count likely doesn't explain the gap.

```
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

|                  | sum_sq      | df   | F        | PR(>F)   |
|------------------|-------------|------|----------|----------|
| C(full_genotype) | 936.483669  | 3.0  | 0.867070 | 0.470188 |
| C(sex)           | 52.938403   | 1.0  | 0.147043 | 0.704380 |
| Residual         | 9720.503943 | 27.0 | NaN      | NaN      |

Figure 6: 2 Way ANOVA F-test (sex:genotype interaction effect left out due to multicollinearity issues arising from small sample size).

A two-way ANOVA model was constructed predicting accuracy with categorical variables sex and genotype. The F-test for genotype main effect significance had a p-value of 0.47, and the sex main effect p-value was 0.70. Thus neither genotype nor sex likely explain the gap.

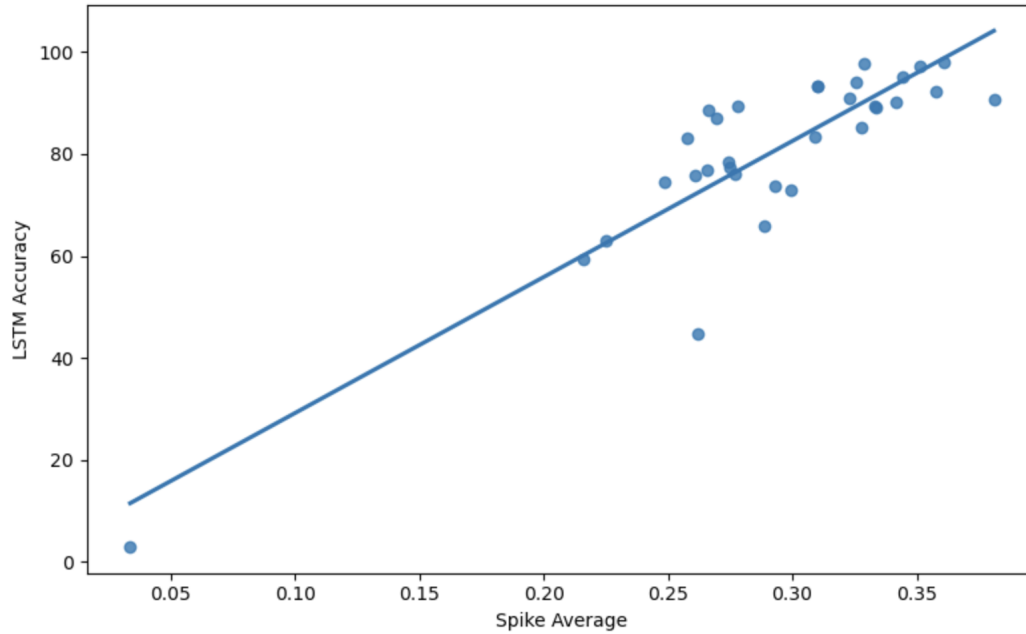


Figure 7: Accuracy vs Spike Average

Spike Average is the average neural firing rate across all neurons and time bins. The p-value of this simple linear regression model's overall significance F-test is  $2.09e-11$ , and its  $R^2$  is 0.78, meaning average firing rate explains 78% of the variation in LSTM accuracy. Average firing rate explains the gap very well, and a possible interpretation for why could be: a higher average firing rate means a more active mouse brain, which means a mouse that is more actively engaged and responding to the visual stimuli, which means their neural firing rates are more correlated with the images and less noisy. Level of mouse engagement with visual stimuli could naturally vary widely across subjects. Average firing rate not only explains the general trend, but also cleanly explains the terrible LSTM accuracy of outlier mouse 757216464.

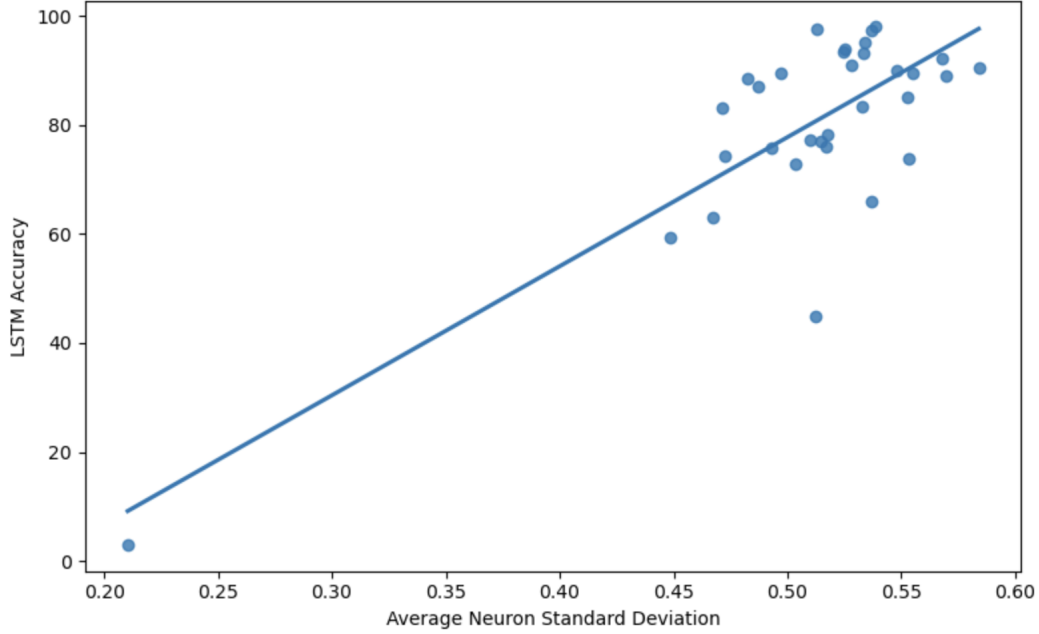


Figure 8: Accuracy vs Average Neuron Standard Deviation (std. dev. for each neuron calculated across time (bins)).

Average Neuron std. dev, is calculated as an average of each neuron’s standard deviation, and is designed to capture the level of variance in neural activity of a mouse. The p-value of this simple linear regression model’s overall significance F-test is  $3.90e-8$ , and its  $R^2$  is 0.64, which is less predictive than average firing rate but still statistically significant. However when a nested F-test is run to see whether average standard deviation is statistically significant predictor of LSTM accuracy once average firing rate has been accounted for, it fails the test with a p-value of 0.33. A possible interpretation is that both average firing rate and average firing rate standard deviation are good measures of mouse engagement with visual stimuli which is a good predictor of LSTM accuracy, but average firing rate is a better measure and highly positively correlated with average standard deviation.

Thus although other potential models combining various predictors have not been tested (other than the two way ANOVA), average firing rate by itself is likely the best predictor of LSTM accuracy among the available metadata considered in this project, as other predictors show little signs of explanatory power.

## 4 Conclusion & Future Directions

The finding that LSTM overfitting is hard to address and that LSTM accuracy can be well predicted by average neural firing rate suggests that there may be signal to noise ratio limit the LSTM is hitting, which is inherent to the data, potentially connected to the mouse’s engagement with the visual stimuli. Further work in data collection should explore how to increase this neural firing average for better performing models. Additionally, the boundaries of simple LSTM-like structures could be explored further on other more complex neural decoding tasks beyond image classification.

## References

AIBS. Allen brain observatory – neuropixels visual coding technical white paper: Overview. Technical report, Allen Institute for Brain Science, 2019. Version 1.0 (October 1, 2019).



Carpenter, Vita, and Pipiras. Benchmarking deep learning architectures for predicting visual stimuli given single neuron spike patterns. *Cognitive Computational Neuroscience*, 2024.